



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 7th**

Subject Name – Modern Compiler Design

Subject Code – BE103701

Faculty Name – Radhika Gokani

CHAPTER -4 Error Recovery

4 Introduction

Error Recovery in Modern Compiler Design

In compiler design, **error recovery** refers to the techniques used by a compiler to **detect errors, report them clearly, and continue parsing** the remaining input so that multiple errors can be identified in a single compilation cycle.

Modern compilers aim not only to find errors but also to **minimize cascading errors** and provide **meaningful, user-friendly messages**.

Objectives of Error Recovery

- To detect syntactic and semantic errors.
- To report errors accurately with useful diagnostic information.
- To recover and continue parsing to find more errors.
- To avoid generating misleading secondary errors.
- To maintain compiler robustness and usability.

Types of Errors Handled

Compilers typically handle:

- Lexical errors (illegal characters)
- Syntax errors (missing symbols, incorrect structure)
- Semantic errors (type mismatches, undeclared variables)
- Logical errors (not handled by compiler but by runtime tools)

Traditional Error Recovery Methods

1. Panic Mode Recovery

- Simplest and widely used.
- On detecting an error, the parser skips input symbols until a “synchronizing token” (like ; or }) is found.
- Ensures quick recovery and prevents infinite loops.
- Used in most modern parser generators.

2. Phrase-Level Recovery

- The parser makes **local corrections** to the input—such as inserting, deleting, or replacing tokens—to allow parsing to continue.
- Can repair minor errors but may introduce spurious modifications.

3. Error Productions

- Special grammar rules are added to the grammar to catch common mistakes.
- Helps produce specific error messages.
- Used when certain errors are predictable.

4. Global (Minimum-Distance) Correction

- Attempts to find the minimum number of changes needed to transform erroneous input into valid input.
- Uses algorithms to compute “edit distance.”
- Very accurate but computationally expensive; rarely used in real compilers.

Modern Error Recovery Techniques

Modern compilers combine classic techniques with advanced diagnostics, including:

1. Intelligent Token Insertion and Deletion

- Uses heuristics and parsing algorithms (LR, LL, GLR) to suggest probable fixes.
- Example: Suggesting missing parentheses or braces.

2. Context-Sensitive Error Messages

- Compilers analyze symbol tables, types, and control flow to produce helpful messages.
- Example:
Instead of “syntax error,” modern compilers say:
"Expected ';' after expression"

3. Look ahead-Based Recovery

- Advanced parsers (GLR, LL(*), PEG) inspect deeper into token streams to identify the best recovery path.

4. Abstract Syntax Tree (AST) Repair

- The parser constructs a **partial AST** even in the presence of errors.
- After parsing, the AST is checked and corrected to continue compilation.
- Used in IDEs for syntax highlighting and autocomplete.

5. IDE-Integrated Recovery

Modern compilers (Clang, GCC, Java Compiler) support:

- Incremental parsing
- Real-time error detection
- Non-terminal fallbacks
- Fault-tolerant parsing for editors

These techniques allow code editors to function smoothly even when the code is incomplete.

4.1 Error Detection & Recovery

Compilers must detect errors in the source program and try to recover from them so that compilation can continue. This process ensures that programmers receive meaningful feedback on multiple mistakes in a single run.

1. Error Detection

Error detection is the process by which a compiler identifies invalid constructs in the source code.

Types of Errors Detected by a Compiler

1. Lexical Errors

- Illegal characters, misspelled tokens.

Example: @value = 10

2. Syntax Errors

- Violations of grammar rules.

Example: missing ;, unbalanced parentheses.

3. Semantic Errors

- Incorrect meaning, type mismatch, undeclared variables.

Example: x = "hello" + 2;

4. Logical Errors

- Not detected by compiler; detected at runtime or by debugging.

Example: wrong algorithm.

Characteristics of Good Error Detection

- Identify the **type** and **location** of error.
- Provide **clear, meaningful messages**.
- Avoid reporting **false or cascading** errors.

2. Error Recovery

Error recovery is the technique used by the compiler to continue parsing the remaining input after detecting an error.

A good error recovery strategy:

- Prevents infinite loops
- Minimizes cascading errors
- Maintains correct context

Major Error Recovery Strategies

1. Panic Mode Recovery

- Simplest and most widely used technique.
- On error, the parser discards tokens until a **synchronizing token** is found (e.g., ;, {, }).
- Fast and safe, but may skip more input than necessary.

Example:

Missing ; in a statement; compiler skips to the next ; or }.

2. Phrase-Level Recovery

- Local correction near the error.
- Compiler inserts, deletes, or replaces tokens to repair input.
- Allows parsing to continue smoothly, but may introduce incorrect fixes.

Example:

Insert missing) or ; automatically.

3. Error Productions

- Special grammar rules to catch common mistakes.
- Gives precise error messages.

Example:

If students commonly forget then in an if statement, an error-production rule can detect that pattern.

4. Global Correction

- Uses algorithms to find the **minimal number of changes** that make input valid.
- Very accurate but computationally expensive.
- Not commonly used in practical compilers.

Error Detection & Recovery in Different Phases**A. Lexical Phase**

- Detects illegal characters, malformed tokens.
- Recovery:
 - Skip invalid characters
 - Replace with valid ones

B. Syntax Phase

- Detects grammar violations.
- Recovery:
 - Panic mode
 - Phrase-level corrections
 - Error productions

C. Semantic Phase

- Detects meaning-related errors (type mismatch, undeclared variables).
- Recovery:
 - Assume default types
 - Ignore erroneous expressions
 - Continue analyzing rest of code

Example of Error Detection & Recovery

Code:

```
int main() {  
    int x = 5  
    printf("%d", x);  
}
```

Compiler output:

error: expected ';' after expression (line 2)

note: inserting ';' and continuing

- Error detected: Missing ;
- Recovery: Phrase-level (insert ;)

4.2 Ad-Hoc and Systematic Methods

Error recovery techniques in compilers can broadly be classified into:

1. **Ad-Hoc Methods**
2. **Systematic Methods**

These methods are used during syntax analysis (parsing) to handle invalid inputs and recover gracefully.

1. Ad-Hoc Methods

Ad-hoc methods are **informal, heuristic-based approaches** used by simple or older compilers. They are easy to implement but may not always give accurate or optimal recovery.

Characteristics

- Simple and quick to implement
- Based on common mistake patterns
- Not backed by formal parsing theory
- May cause additional errors (cascading)
- Used mostly in early compilers or basic parsers

Common Ad-Hoc Techniques

a) Panic Mode Recovery

- Skip tokens until a synchronizing symbol (like ; or }) is found.
- Very common and safe.
- But may skip more input than necessary.

b) Inserting or Deleting Tokens

- Parser tries to fix errors by:
 - inserting missing symbols (;,), })
 - deleting unexpected tokens
- Simple but may change logic of the program.

c) Replacing Tokens (Substitution)

- Replace erroneous tokens with likely correct ones.

d) Default Values

- Assign a default type or value during semantic errors to continue analysis.

Example (Ad-Hoc)

Input:

```
if (a > b { c = a; }
```

Compiler may guess user intent and insert missing) after b.

2. Systematic Methods

Systematic methods are **formal, theoretically sound techniques** based on parsing algorithms and grammar theory. They provide more **accurate, structured, and predictable** error recovery.

Characteristics

- Based on formal grammar rules
- More robust and reliable
- Used in modern parser generators (e.g., YACC, Bison, ANTLR)
- Can generate high-quality error messages

Common Systematic Techniques

a) Error Productions

- Grammar includes special rules to catch common patterns of mistakes.
- Helps give clear messages for known error-prone constructs.

Example:

```
stmt → if expr then stmt
```

```
stmt → if expr stmt // Error production: missing 'then'
```

b) Global Correction

- Computes the **minimum number of insertions, deletions, or replacements** to convert erroneous input into valid input.
- Very accurate but computationally expensive.
- Rarely used in practical compilers.

c) Phrase-Level Recovery (Formal Version)

- Parser uses grammar-based strategies to make small, controlled fixes near error points.
-

Difference Between Ad-Hoc and Systematic Methods

Ad-Hoc Methods	Systematic Methods
Heuristic, informal	Formal, theory-based
Easy to implement	More complex
Fast but less accurate	Accurate and reliable
May miss some errors	Catches errors systematically
Common in simple compilers	Used in modern professional compilers

Example to Show Both Approaches

Code:

```
while (x < 10 x++;
```

Ad-Hoc Approach

Insert missing) automatically, assume:

```
while (x < 10) x++;
```

Systematic Approach

Grammar rule catches error:

```
expr → expr relop expr ')' // error production
```

Compiler provides a structured message:

error: expected ')' before 'x'
